

CHAPTER 3: RESEARCH METHODOLOGY AND PROPOSED SYSTEM

3.1 Introduction

This chapter describes the research methodology used in the design and development of the Cloud-Based Examination Script Management System for the Computing Sciences Department of Koforidua Technical University. It explains the approach followed during the development process and highlights the major activities carried out to achieve the objectives of the project. The chapter also discusses how the selected methodology supported the successful implementation of the proposed system.

The chapter further explains the software development methodology adopted for the project, including the reasons for selecting it over other development approaches. It outlines the different phases involved in the development process and describes the activities performed during each phase, such as requirement gathering, system design, implementation, testing, deployment, and maintenance. The methodology provided a structured and flexible approach that ensured continuous improvement throughout the project.

In addition, the chapter presents the proposed system and its overall architecture. It discusses the major components, system features, development tools, and technologies used in building the application. The proposed system was designed to support secure examination script upload, cloud storage integration, PDF compression, and efficient retrieval of academic documents through a centralized web platform.

Furthermore, the chapter includes various UML models used to represent the structure and behavior of the proposed system. These models include the context diagram, sequence diagram, class diagram, and activity diagram, which illustrate the interactions between users and the system. The chapter also examines the strengths and possible limitations of the proposed solution to provide a clear understanding of its capabilities and operational challenges.

3.2 Software Development Methodology

3.2.1 Introduction to Agile Methodology

The Agile software development methodology was adopted for the development of this project. Agile is an iterative and incremental software development approach that focuses on flexibility, collaboration, continuous improvement, and rapid delivery of functional software components. Agile encourages frequent interaction with stakeholders and allows developers to respond quickly to changing requirements throughout the development process.

Unlike traditional plan-driven methodologies such as the Waterfall model, Agile supports continuous testing, regular feedback, and iterative development through short development cycles known as sprints. This makes Agile suitable for projects where requirements may evolve during development.

3.2.2 Justification for Choosing Agile Methodology

The Agile methodology was selected for this project because of its flexibility and ability to support continuous improvement during development. The Cloud-Based Examination Script Management System required regular adjustments and feature enhancements throughout the development process, making Agile more appropriate than rigid plan-driven models.

The following reasons justify the use of Agile methodology for this project:

1. Flexibility in Requirement Changes

The project requirements evolved during development as new ideas and user feedback emerged. Agile supports changes at any stage of development without affecting the overall progress of the project.

2. Iterative Development

The project was developed in small phases known as sprints. Each sprint delivered functional features such as user authentication, script upload, cloud storage integration, PDF compression, notifications, and access control.

3. Continuous Testing and Feedback

Agile allowed testing to be performed continuously after each sprint. This helped identify errors early and improved system reliability.

4. Improved User Involvement

The methodology encouraged regular feedback from lecturers, departmental administrators, and potential users, ensuring that the developed system met user expectations and academic requirements.

5. Faster Delivery of Functional Components

Agile enabled the development of working features incrementally, allowing portions of the system to become operational before the entire project was completed.

6. Risk Reduction

By dividing the project into smaller manageable phases, risks associated with system failure or major design problems were minimized.

3.2.3 Agile Software Development Process

The Agile development process used in this project consisted of the following phases:

1. Requirement Gathering and Planning
2. System Design
3. Development and Implementation
4. Testing and Debugging
5. Deployment
6. Maintenance and Improvement

The development activities were organized into iterative sprints, with each sprint focusing on specific features of the system.

3.3 Activities Performed in Each Agile Phase

3.3.1 Requirement Gathering and Planning

This phase involved identifying the functional and non-functional requirements of the proposed system. Information was gathered through literature review, observation of existing document management systems, and analysis of the challenges associated with examination script management in educational institutions.

The following activities were carried out:

1. Identification of system users and stakeholders.
2. Definition of project objectives and scope.
3. Collection of functional requirements such as:
 1. User registration and login.
 2. Examination script upload.
 3. PDF compression and optimization.
 4. Cloud storage integration.
 5. Search and retrieval of scripts.
 6. Automated notifications.

7. Time-based access control.
4. Collection of non-functional requirements such as:
 1. Security.
 2. Reliability.
 3. Performance.
 4. Scalability.
 5. Usability.
5. Planning of project timeline and sprint activities.

3.3.2 System Design

The system design phase focused on creating the architecture and models of the proposed system.

The following activities were carried out:

1. Designing the database schema using MySQL.
2. Designing the web application user interface.
3. Creating API structures for communication between frontend and backend.
4. Designing UML diagrams including context, sequence, class, and activity diagrams.
5. Designing authentication and authorization workflows.
6. Designing PDF compression and cloud upload processes.
7. Designing notification and access control mechanisms.

3.3.3 Development and Implementation

This phase involved the actual coding and implementation of the system components.

The following technologies were used:

1. React for frontend development.
2. Next.js and Express.js for backend API development.
3. MySQL for database management.
4. Cloud storage integration services.

5. Postman for API testing.
6. WebStorm IDE for development.

The following modules were implemented:

1. User Authentication Module.
2. Examination Script Upload Module.
3. PDF Compression Module.
4. Cloud Storage Module.
5. Script Retrieval and Search Module.
6. Notification and Alert Module.
7. Access Control and Authorization Module.
8. Administrative Management Module.

3.3.4 Testing and Debugging

Testing was performed continuously throughout development to ensure that all features worked correctly.

The following testing methods were used:

Unit Testing

Each module was tested independently to verify correct functionality.

Integration Testing

Communication between frontend, backend APIs, cloud storage services, and database components was tested.

System Testing

The complete system was tested to ensure that all modules worked together correctly.

User Acceptance Testing

Potential users such as lecturers and departmental administrators tested the system to evaluate usability and functionality.

The following tools were used during testing:

1. Postman for API testing.

2. Browser developer tools.
3. MySQL Workbench for database verification.

3.3.5 Deployment

After successful testing, the application was deployed for demonstration and evaluation.

The following activities were carried out:

1. Backend server setup.
2. Database deployment.
3. Cloud storage configuration.
4. Frontend deployment.
5. Final system configuration.

3.3.6 Maintenance and Improvement

Maintenance activities involved fixing bugs, improving performance, and preparing recommendations for future enhancements.

Possible future improvements include:

1. Integration with university academic systems.
2. AI-based document classification and indexing.
3. Mobile application support.
4. Real-time collaboration and moderation features.
5. Expansion to other departments within the university.

3.4 Proposed System

3.4.1 Introduction of the Proposed System

The proposed system is a Cloud-Based Examination Script Management System designed for the Computing Sciences Department of Koforidua Technical University. The system enables lecturers and administrators to upload, store, retrieve, and manage examination scripts electronically through a centralized web platform.

The system integrates PDF compression technology to reduce document sizes before cloud storage, thereby improving storage efficiency and upload performance. It also provides role-based and time-based access control to ensure that only authorized users can access examination scripts within approved periods.

The proposed system is intended to reduce the challenges associated with manual paper-based storage, improve document accessibility, enhance security, and support long-term preservation of academic records.

3.5 Models of the Proposed System

3.5.1 Context Model

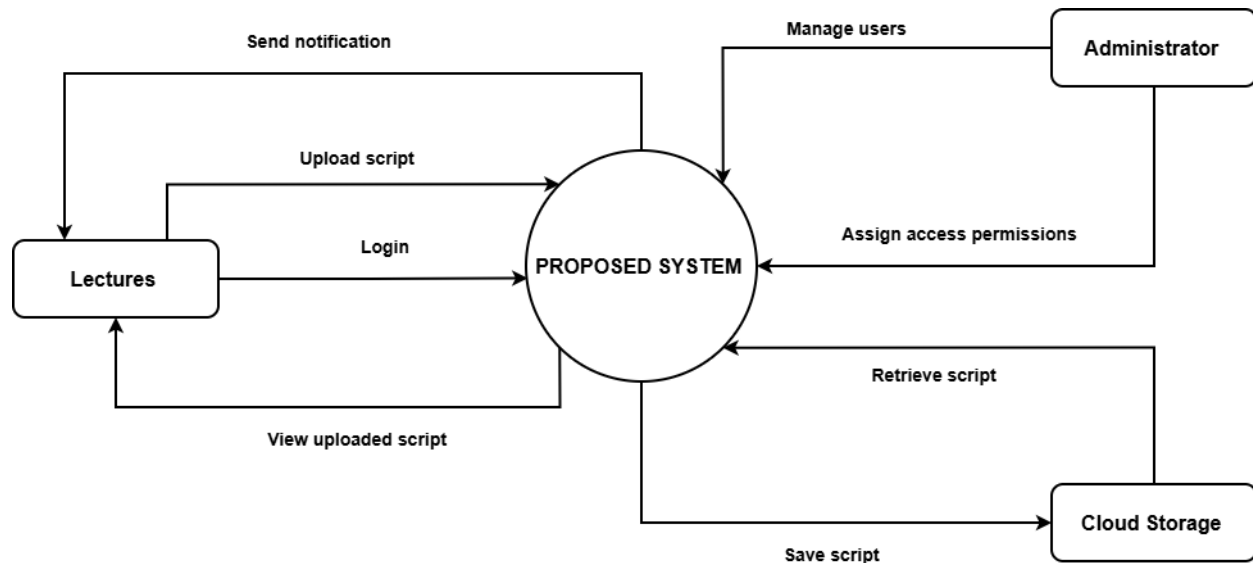


Figure 3.1

Description

The context model for the system consists of several external entities that interact with the main platform to support its operations and services. Lecturers interact with the system by uploading examination scripts, viewing uploaded documents, and receiving notifications related to submission deadlines. Department administrators manage users, monitor uploaded scripts, assign access permissions, and oversee system activities.

Cloud storage services provide remote storage facilities for uploaded examination scripts and ensure secure file preservation and retrieval. Notification services enable the delivery of alerts and reminders to lecturers and administrators through email or in-system notifications. The database

server stores and manages all system data securely, including user information, metadata, access logs, and examination script records.

3.5.2 Interaction Model (Sequence Diagram)

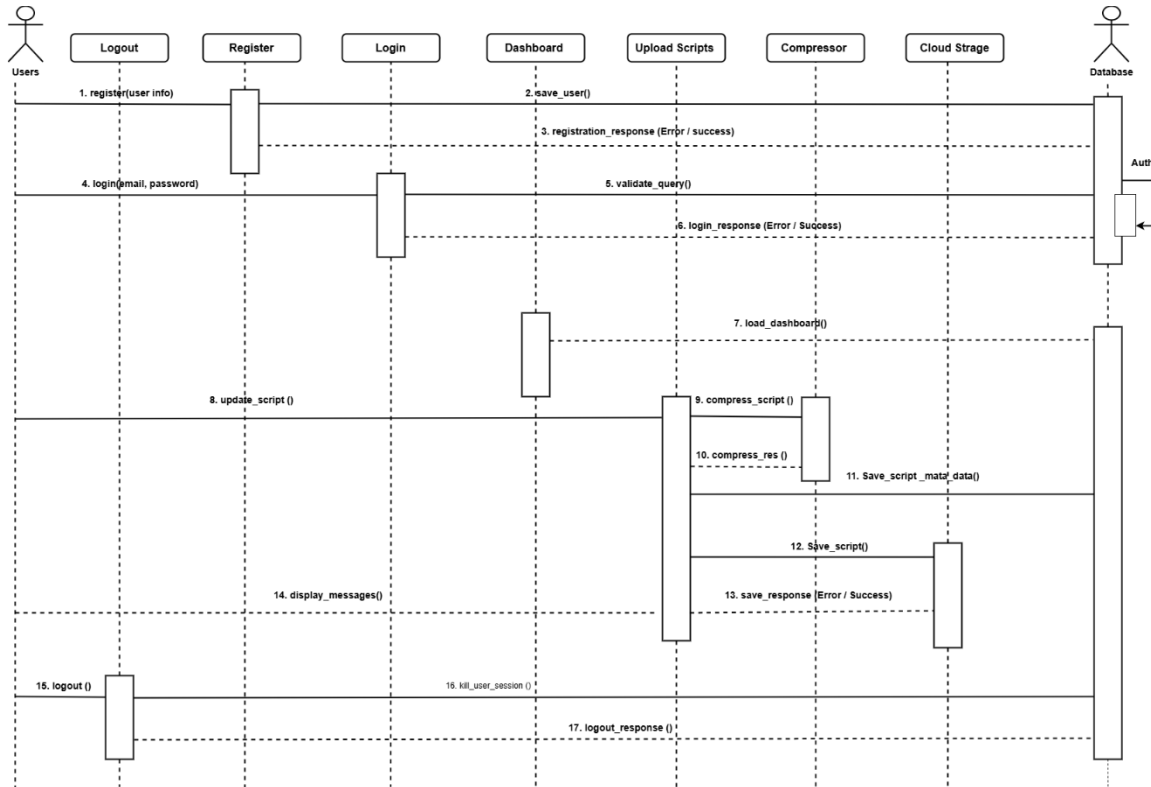


Figure 3.2

Description

The sequence flow of the system begins when the user opens the web application and logs into the system to access its services. After successful authentication, the user selects the option to upload an examination script and enters the required script details such as course code, semester, and academic year.

The user then uploads the PDF document, after which the system validates and compresses the file to optimize storage usage. Once processing is completed successfully, the file is uploaded to cloud storage while the related metadata is stored in the MySQL database.

After successful storage, the system sends notifications to administrators or relevant users. Finally, the user receives a confirmation message indicating that the upload process was completed successfully.

3.5.3 Structural Model (Class Diagram)

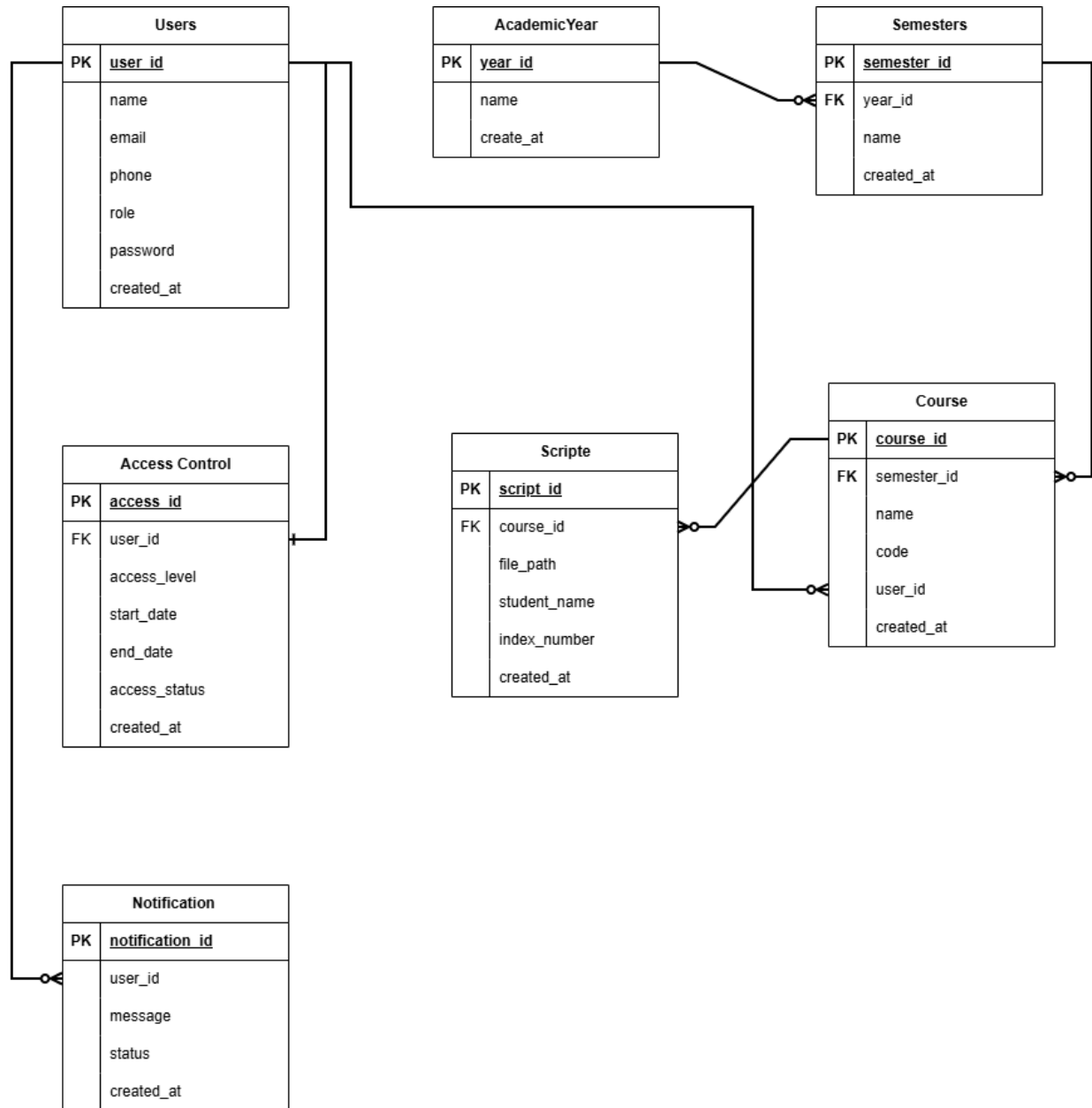


Figure 3.3

Description

The class structure of the system is composed of several main classes that work together to support examination script management functionalities. The User class stores user-related information such as UserID, Name, Email, Role, and Password for authentication and authorization purposes.

The ExaminationScript class manages uploaded examination documents and contains attributes including ScriptID, CourseCode, AcademicYear, Semester, FilePath, and UploadDate to support proper organization and retrieval of scripts.

The Notification class handles communication between the system and users by storing notification information such as NotificationID, Message, Status, and Timestamp. The AccessControl class manages role-based and time-based permissions using attributes such as AccessID, UserRole, AccessStartDate, and AccessEndDate.

Finally, the CloudStorage class manages cloud file information including StorageID, FileName, FileSize, and UploadStatus to support secure and efficient cloud storage operations.

3.5.4 Behavioral Model (Use Case Diagram)

The use case diagram illustrates the interaction between the lecturers, administrator, system database, and notification service within the Cloud-Based Examination Script Management System. The system is designed to support secure examination script upload, access management, notification handling, and document retrieval, with all activities connected to a centralized database.

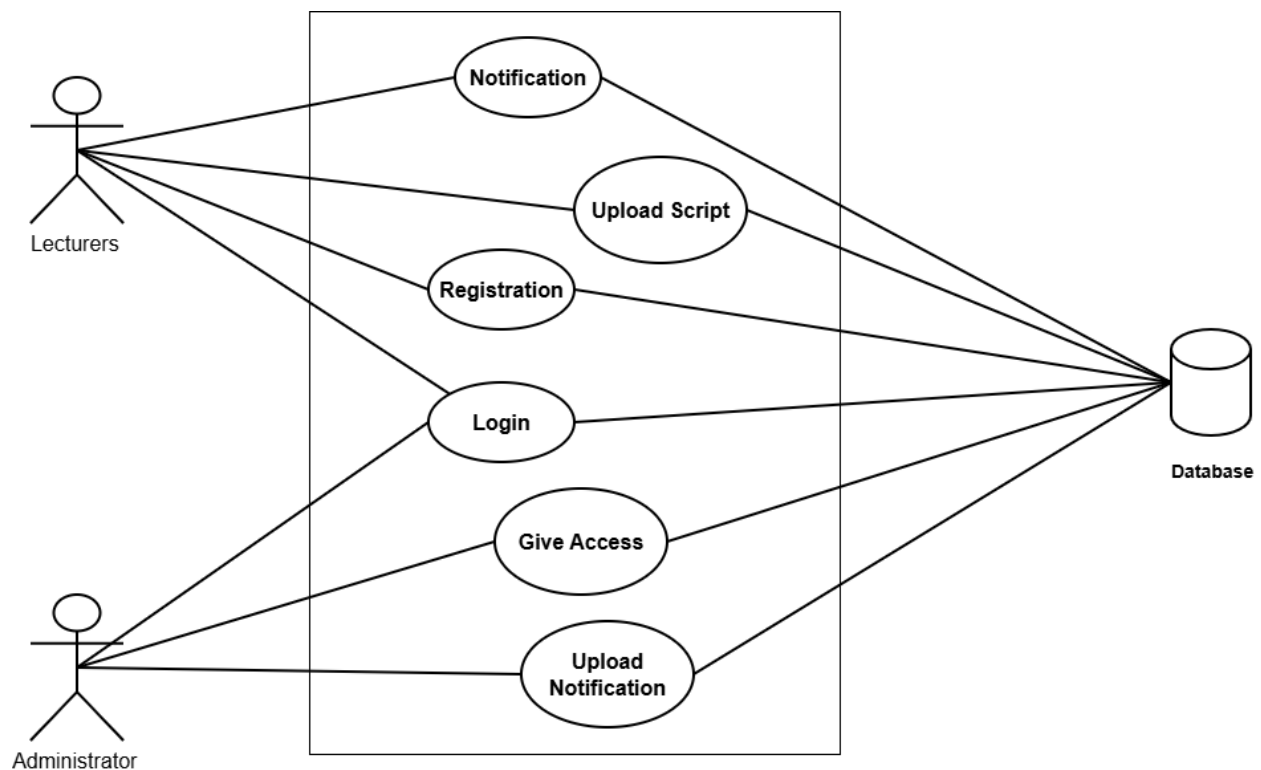


Figure 3.4

Description

The behavioral model of the Cloud-Based Examination Script Management System is represented using a use case approach to show the interactions between lecturers, administrators, the database, and notification services. The process begins when lecturers register and log into the web application to gain secure access to the platform. The administrator also logs into the system and grants access permissions to authorized lecturers.

After authentication, lecturers upload examination scripts by entering the required document details and selecting the PDF file for submission. The system validates and compresses the uploaded file before storing it securely in cloud storage. Metadata associated with the uploaded script is saved in the centralized database for easy management and retrieval.

Once the upload process is completed, notifications are automatically sent to the administrator to inform them of newly uploaded scripts, while lecturers receive confirmation feedback regarding successful uploads. Authorized users can later retrieve examination scripts using the search and retrieval functionality. Throughout the entire process, all operations and records are connected to and managed through the database to ensure security, accessibility, and proper document management.

3.6 Features of the Proposed System

The proposed system includes the following features:

1. User registration and authentication.
2. Examination script upload functionality.
3. PDF compression and optimization.
4. Cloud-based document storage.
5. Secure script retrieval system.
6. Automated notifications and reminders.
7. Role-based access control.
8. Time-based authorization.
9. Search and indexing functionality.
10. Secure backend APIs.
11. User-friendly web interface.
12. Centralized database management.

13. File backup and recovery mechanisms.

3.7 Development Tools and Development Environment

The following development tools and technologies were used in the implementation of the proposed system.

3.7.1 React

React was used for developing the frontend user interface. It supports responsive and interactive web application development.

Advantages

1. Reusable components.
2. Fast rendering performance.
3. Responsive user interface.
4. Large community support.

3.7.2 Next.js and Express.js

Next.js and Express.js were used for backend API development and server-side processing.

Advantages

1. Fast API development.
2. Scalable architecture.
3. Efficient routing and middleware support.
4. Easy integration with databases and cloud services.

3.7.3 MySQL Database

MySQL was used as the database management system for storing user data, script metadata, notifications, and access control information.

Advantages

1. Reliable data storage.

2. Structured data management.
3. High performance.
4. Secure database operations.

3.7.4 Postman

Postman was used for API testing and debugging.

Uses

1. Testing API endpoints.
2. Verifying request and response data.
3. Debugging backend services.
4. Monitoring API performance.

3.7.5 Cloud Storage Services

Cloud storage services were used to store uploaded examination scripts securely.

Uses

1. Remote document storage.
2. File backup and recovery.
3. Secure file access.
4. Scalable storage management.

3.7.6 WebStorm IDE

WebStorm IDE was used as the primary development environment for coding and managing the project.

Advantages

1. Intelligent code completion.
2. Debugging tools.
3. Integrated terminal.

4. Git integration.
5. Improved developer productivity.

3.8 Review of the Good Features of the Proposed System

The proposed system offers several advantages that improve examination script management.

1. Centralized cloud storage improves document organization.
2. PDF compression reduces storage consumption.
3. Automated notifications improve submission compliance.
4. Secure authentication protects academic records.
5. Role-based access control improves confidentiality.
6. Fast retrieval improves efficiency in accessing scripts.
7. Backup and recovery mechanisms reduce data loss risks.
8. Agile development ensured continuous improvement and testing.
9. Reduced dependence on paper-based storage systems.

3.9 Review of the Bad Features of the Proposed System

Despite its advantages, the proposed system has some limitations.

1. Dependence on internet connectivity.
2. Requirement for digital literacy among users.
3. Dependence on third-party cloud services.
4. Possible storage subscription costs.
5. Initial setup and training requirements.
6. Potential cybersecurity threats if not properly maintained.

These limitations may affect system accessibility, operational cost, and long-term maintenance requirements.

3.10 Summary of the Chapter

This chapter presented the methodology and proposed system for the Cloud-Based Examination Script Management System. The Agile software development methodology was selected due to its flexibility, iterative nature, and ability to support continuous testing and feedback.

The chapter discussed the phases of the Agile process and the activities performed during each phase of development. It also presented the proposed system architecture, UML models, system features, development tools, and development environment.

The strengths and limitations of the proposed system were also analyzed. Overall, the proposed system is designed to improve examination script management through secure cloud storage, PDF compression, automated notifications, and efficient retrieval of academic records.